REQ2 Mining in the Mines Design Rationale

Introduction

REQ2 introduces the Mines map, several ways to travel between maps, mineable objects, environmental hazards, and resources. The central design question is therefore not only how to make each individual feature work, but how to prevent the number of actions, terrain types, and conditional checks from growing every time a new tool, map, or hazard is added.

This rationale compares two plausible designs for each major REQ2 area. Design 1 is the selected implementation. Design 2 is a reasonable alternative, but it was not selected because it creates stronger coupling, repeats logic, or conflicts with the engine abstraction that best represents the game object. The discussion links each decision to the REQ2 UML diagram and source code so that the rationale, diagram, and implementation can be assessed together.

A Travel between maps

REQ2 requires ladders, teleportation circles, and a portable Alien Device. Although their side effects differ, all three move an actor to a selected Location. The chosen design separates the shared player action from the rule owned by each travel mechanism.

Design 1 Selected: one TeleportAction with the Teleportable interface

Create a Teleportable interface with teleport(Actor, Location, GameMap). Ladder, TeleportationCircle, and AlienDevice implement the interface. TeleportAction stores a Teleportable reference and delegates execution to it. Each mechanism creates a TeleportAction only when it is legally usable.

Advantages	Trade-offs
- Avoids three near-identical action classes. TeleportAction contains the common Action integration once. - Supports Open-Closed Principle: a future portal, vehicle, or new-map device can implement Teleportable without modifying TeleportAction. - Supports Dependency Inversion Principle: TeleportAction depends on Teleportable, not on Ladder, TeleportationCircle, or AlienDevice. - Keeps special effects cohesive: Ladder owns the fall chance, TeleportationCircle owns arrival fire, and AlienDevice owns malfunction behaviour. - Allows the device to expose several destinations through its destination list, matching the requirement for future maps.	- The interface receives a Location and GameMap, so each implementation must validate a configured destination before movement. - The generic action menu description is less specialised than a separate action class unless the mechanism supplies a clear display name. - A very different future travel rule may need extra configuration rather than expanding the small interface with unrelated parameters.

Design 2 Not selected: separate action class and type checks for every mechanism

Create LadderAction, TeleportationCircleAction, and AlienDeviceAction. Each execute method would check its concrete mechanism type and perform movement plus the mechanism's side effect itself.

Advantages	Trade-offs
- A first implementation is direct because each action can be named specifically for its mechanism. - All code for a simple mechanism appears in one action class, which can feel easy to find initially.	- Repeats Action execute, destination storage, movement, and menu plumbing across classes, violating DRY. - A new mechanism requires a new Action class and often a change to menu logic, increasing class coupling. - Mechanism rules move away from the object that owns them. For example, circle fire would be split between Circle and CircleAction. - Type checks make the design harder to extend and weaken polymorphism.

Discussion and selected design

Design 1 was selected because the shared behaviour is genuinely stable: every travel option presents an Action and ultimately delegates movement through the configured destination. The differences are domain rules, not different action lifecycles. Ladder extends Ground because use is permitted only while standing on its tile; TeleportationCircle also extends Ground for the same reason; AlienDevice extends Item because it is carried. This uses the engine as intended rather than simulating a carried device as a map tile. StackQueueValleyWorld configures paired ladder and circle destinations before play, while AlienDevice keeps a list of destinations. This prevents temporal connascence: the destination relationship is established in one setup location rather than being repeatedly reconstructed by player actions.

How the selected travel design meets REQ2

Mechanism	Class responsibility	REQ2 result
Ladder	Ladder implements Teleportable and performs a 50 percent fall check before moving the actor.	The farmer receives 25 damage on a fall but still arrives at the configured destination.
Teleportation Circle	TeleportationCircle implements Teleportable and creates Fire at each adjacent arrival location.	The arrival tile remains safe while neighbouring locations burn for the configured circle-fire values.
Alien Device	AlienDevice is portable, provides an action per configured destination, and chooses a valid current-map location on malfunction.	The farmer can use the device anywhere and it has the required 50 percent malfunction branch.

B Breaking objects in the mines

Rocks, copper nodes, and geodes are all breakable, but REQ2 explicitly says that later tools may also break things and later objects may become breakable. The design must therefore avoid treating Pickaxe, Rock, CopperNode, and Geode as a closed list.

Design 1 Selected: BreakAction with Breakable and Breaker capability interfaces

Create Breakable with breakEntity(Actor, Location, GameMap), Breaker as the tool capability, and one BreakAction that delegates to the selected Breakable target. Pickaxe implements Breaker; Rock, CopperNode, and Geode implement Breakable.

Advantages	Trade-offs

Advantages	Trade-offs
- Supports Open-Closed Principle: a Hammer can implement Breaker and a treasure chest can implement Breakable without changing existing action code. - BreakAction has one responsibility: invoke the selected target's breaking contract and return its result. - The target owns its unique reward and hazard, avoiding a large if-else or switch statement in BreakAction. - The design uses the engine Action class for a player choice rather than creating an unnecessary Behaviour. - The Farmer only needs a Pickaxe Item in its inventory; it does not need mining logic in playTurn.	- The marker interface is intentionally minimal, so readers must use the class name and Javadoc to understand its meaning. - Each breakable object checks the actor inventory before exposing or executing the action. This small repeated policy is a trade-off for preserving each object's engine-specific allowableActions context.

Design 2 Not selected: hard-code Pickaxe and object types in Farmer or BreakAction

Put a large conditional statement in Farmer.playTurn or BreakAction that tests instanceof Pickaxe and then tests whether the target is Rock, CopperNode, or Geode before applying the appropriate effect.

Advantages	Trade-offs
- Uses fewer initial classes and may be quicker for a one-off prototype. - All current mining cases are visible in a single method.	- Violates Open-Closed Principle: every new tool or target requires editing the central conditional statement. - Violates Single Responsibility Principle: Farmer or BreakAction would own tool permission, target classification, rewards, explosions, charging, and movement. - Encourages instanceof and downcasting, coupling high-level gameplay flow to concrete mine objects. - Makes independent testing of a rock or geode outcome harder because their logic is not local to the object.

Discussion and selected design

Design 1 was selected because it models two independent facts: a tool is capable of breaking, and a target is capable of being broken. The separate Breaker and Breakable contracts avoid false assumptions about either side. For example, a future hammer may break rocks but not a later fragile object, or a future chest may be breakable but have a different outcome. BreakAction depends only on Breakable, satisfying Dependency Inversion. The concrete objects remain responsible for their own state transition and reward, which keeps the code cohesive and prevents the action layer from becoming a second game-rule engine.

C Representing mine targets and rewards

REQ2 intentionally gives geodes different interaction rules from rocks and copper nodes. The selected design uses the existing engine abstractions to represent this difference instead of forcing every mine target into the same class hierarchy.

Design 1 Selected: obstructing objects are Ground; portable geodes and rewards are Item

Rock and CopperNode extend Ground and reject actor entry until broken. Geode, Stone, CopperOre, Diamond, and AlienDevice extend Item. Geode additionally implements Breakable so it can be broken whether it is carried or on the ground.

Advantages	Trade-offs
- Matches the engine semantics directly: Ground occupies a map tile, while Item can be collected and kept in inventory. - Avoids artificial state flags such as isCarried on a terrain object. - Keeps rock and copper-node path blocking simple through canActorEnter returning false. - Allows Geode to use the engine's existing inventory and ground-item action pathways. - Maintains Liskov Substitution Principle because every terrain remains usable as Ground and every reward remains usable as Item.	- Breaking behaviour is implemented across both Ground and Item subclasses, so the shared Breakable contract is essential to keep the action consistent. - Geode must decide whether to remove itself from inventory or location before resolving its outcome.

Design 2 Not selected: represent all mine targets as Ground with a carried-state flag

Make Geode extend Ground like Rock and CopperNode, then add a flag and special movement code so a ground geode can be picked up and held by the farmer.

Advantages	Trade-offs
- Places all mine targets beneath one concrete engine superclass. - May initially appear to simplify the UML because the targets share one visible parent.	- Conflicts with the engine's Item inventory model and introduces artificial transfer logic. - A Ground object carrying inventory state has two unrelated responsibilities, weakening cohesion. - Requires more special cases when the geode is broken from inventory and when it is displayed on the map. - Makes future portable mineable items more difficult because they cannot reuse Item behaviour naturally.

Discussion and selected design

Design 1 was selected because it is the simplest representation that preserves the REQ2 distinction. Rock and CopperNode are barriers first, so Ground is the correct abstraction. Geode is a collectable object first, so Item is the correct abstraction. The common operation is provided by Breakable, not by forcing common inheritance. This is a practical application of KISS and interface segregation: share only the capability that is common, while retaining the engine base class that describes how each object behaves in the world.

D Fire and charged ground

Teleportation circles and geodes both create fire, but with different lifespans, burn durations, and damage values. Copper nodes also affect nearby terrain by charging any ground capable of holding a charge. The selected design makes these effects reusable without replacing the original terrain.

Design 1 Selected: reusable Fire Item with a burn status and a focused Chargeable interface

Fire is a non-portable Item placed on an existing Location. Its tick method adds or refreshes a burning status on an actor standing on the tile and removes itself after its configured lifespan. Puddle implements Chargeable, and CopperNode charges adjacent ground through the interface rather than checking directly for Puddle.

Advantages	Trade-offs

Advantages	Trade-offs
- One Fire class supports both circle fire and geode fire by receiving lifespan, duration, and damage parameters. - Fire coexists with Dirt, Puddle, or another terrain because it is an Item rather than a replacement Ground subclass. - Keeping burn lifecycle inside Fire prevents duplicated timer and damage logic in TeleportationCircle and Geode. - Chargeable supports future electrified terrain without modifying CopperNode. - The design reduces connascence: circle and geode only agree on Fire constructor values, not on copied tick logic.	- A status effect introduces lifecycle rules that must be tested: application, refresh, damage, expiry, and fire removal. - Parameterized construction requires clear constants so that the circle values and geode values do not become unexplained magic numbers. - If many independent damage-over-time effects are added later, the private burning-status implementation should be promoted to a reusable status class.

Design 2 Not selected: create special burning terrain subclasses and check specifically for Puddle

Replace each affected Ground with a BurningDirt, BurningPuddle, or similar subclass. In CopperNode, use instanceof Puddle before changing the charged state.

Advantages	Trade-offs
- The visible terrain class would immediately show that a location is burning. - An initial puddle-only implementation is short because the current requirement names Puddle.	- Creates a combinatorial growth of terrain classes as every terrain may need a burning variant. - Replacing terrain risks losing its existing state, especially a charged puddle or later terrain data. - CopperNode becomes coupled to Puddle and must change whenever new chargeable terrain is introduced. - Repeated timer and burn logic across burning terrain subclasses violates DRY.

Discussion and selected design

Design 1 was selected because fire is an overlay hazard, not a new form of soil. Modelling it as a non-portable Item lets it remain on the location with the original Ground. The burn effect remains active for the specified duration after application, and standing on fire refreshes that effect. Similarly, Chargeable captures the only fact CopperNode needs to know. Puddle owns the response to becoming charged and applies its 20 damage per turn. These decisions provide high cohesion and avoid concrete type knowledge in the node or circle classes.

Alignment with UML diagram and implementation

The REQ2 UML diagram and the implementation use the same public design names. Ladder and TeleportationCircle generalise Ground and realise Teleportable. AlienDevice generalises Item and realises Teleportable. Rock and CopperNode generalise Ground and realise Breakable. Geode generalises Item and realises Breakable. Pickaxe generalises Item and realises Breaker. Puddle generalises Ground and realises Chargeable. TeleportAction and BreakAction generalise the engine Action class and depend on their respective interface contracts.

The class diagram represents public architecture. Fire.BurningStatus is private to Fire and is therefore an implementation detail rather than a separate public class in the UML. StackQueueValleyWorld is responsible for creating the Mines map, configuring paired ladders and circles, placing the Alien Device, and populating

mine resources. This maps the static design to the setup code that makes the feature observable when the application runs.

Conclusion

Design 1 was selected in all four comparisons because it uses polymorphism and the supplied engine extension points to contain change. TeleportAction and BreakAction are small, reusable actions; interfaces express the four independent capabilities; and concrete objects own their unique game rules. The alternative designs are simpler only at the first implementation, but they concentrate domain rules in type checks and duplicate engine integration. The selected design is therefore more maintainable, more testable, and more credible for later requirements involving additional maps, travel mechanisms, tools, mineable objects, terrain, and hazards.